UE24CS242B: OPERATING SYSTEMS

Operating Systems Banana Problem 3

By

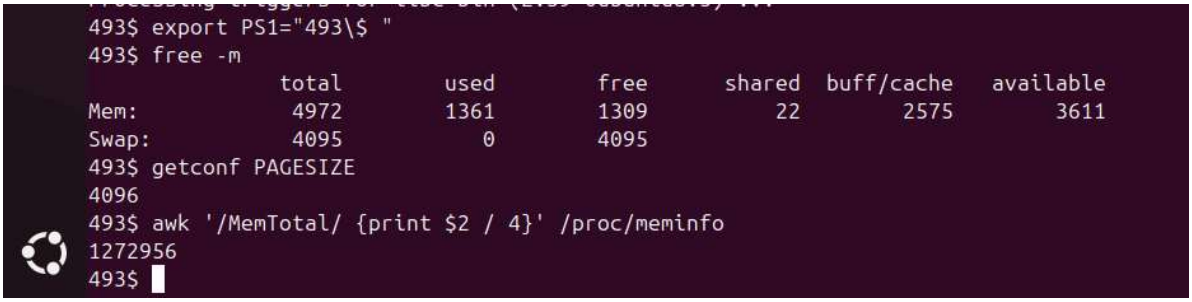
NAME: Tanmayi Nagabhairava

SECTION: I

SRN: PES1UG24CS493

Memory Management

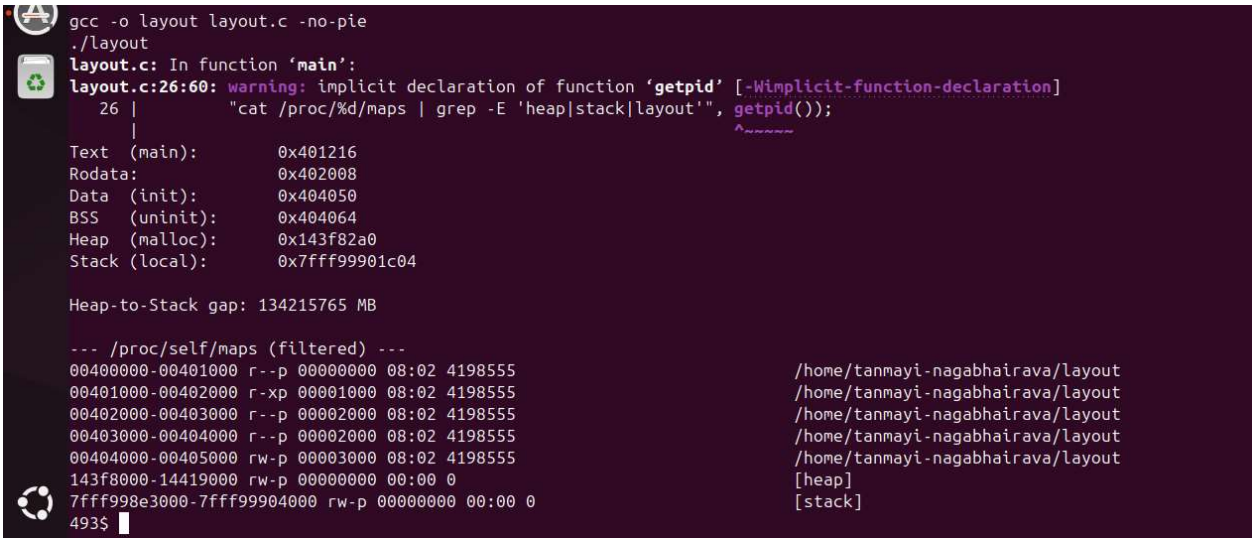
Section 0



Q0.1: If your VM has 2048 MB of RAM and the page size is 4096 bytes, how many physical page frames exist? Show your arithmetic.

- **Calculation:**
  1. Convert RAM to KB: 2048 x 1024 = 2,097,152 KB
  2. Page Size: 4096 bytes = 4 KB
  3. Total Frames: 2,097,152 / 4 = 524,288 **frames**.

Section 1



Q1.1: List the six addresses from lowest to highest. Does the ordering match the diagram? Verify: Text < Rodata < Data < BSS < Heap ≪ Stack.

- **Answer:** Based on my run, the addresses are:
  1. **Text:** `0x401216`
  2. **Rodata:** `0x402008`
  3. **Data:** `0x404050`
  4. **BSS:** `0x404064`
  5. **Heap:** `0x143f82a0`
  6. **Stack:** `0x7fff99901c04`
- **Verification:** Yes, the ordering matches the diagram perfectly. The code and globals are at the bottom, the heap is above them, and the stack is way up in high memory.

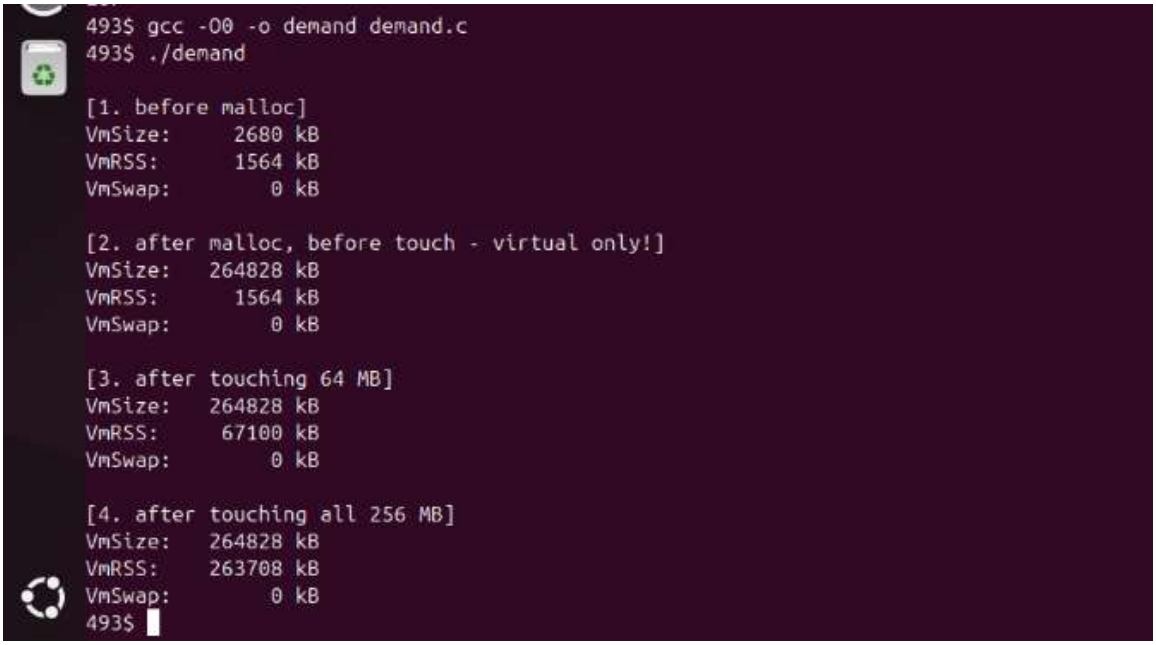
Q1.2: In the filtered `/proc/self/maps` output, what permissions does `[heap]` have? Why `rw-p` instead of `rwxp`? What could go wrong if heap memory were executable?

- **Answer:** The heap has **`rw-p`** permissions (Read, Write, Private). It is not executable (**`x`**) as a security feature called **NX (No-Execute)**. If the heap were executable, an attacker could use a buffer overflow to inject malicious machine code into a variable and execute it, allowing them to take control of the program.

Q1.3: Compile again with PIE: `gcc -o layout_pie layout.c` and run it twice. Do the addresses move between runs? Look up ASLR—why is this a security feature?

- **Answer:** Yes, when compiled with PIE, the memory addresses change every time the program is executed. **ASLR (Address Space Layout Randomization)** is a security feature because it makes the memory locations of the stack, heap, and libraries unpredictable. This prevents attackers from using "hardcoded" memory addresses in their exploits, making it much harder to perform successful buffer overflow or code-injection attacks.

Section 2



Q2.1: After malloc but before any touch, how much did VmSize go up? How much did VmRSS go up? Explain the difference in one sentence.

- **Answer:** According to my output, **VmSize** increased by roughly 262,148 KB (256 MB), while **VmRSS** stayed exactly the same (1564 KB). This is because `malloc` only allocates virtual address space (the OS's promise to provide memory later), but the kernel does not assign actual physical RAM until the memory is accessed.

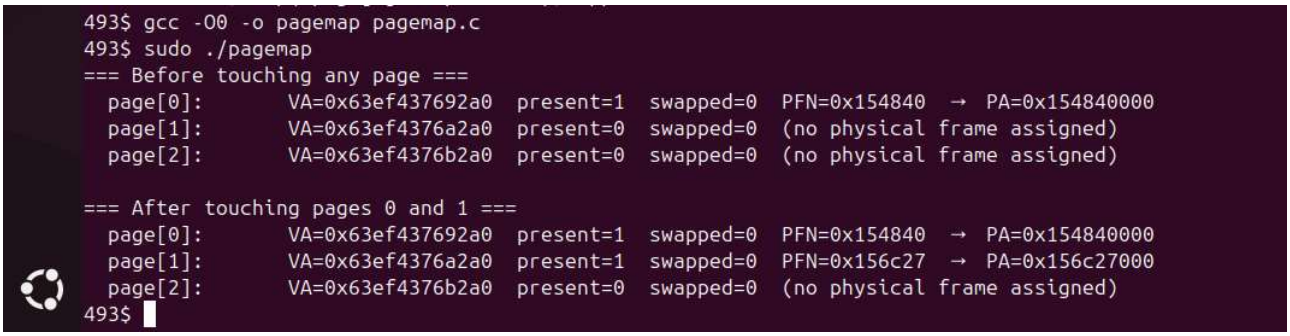
Q2.2: Run `perf stat -e page-faults ./demand`. How many page faults did you get? Does it roughly match 256 MB / 4 KB = 65536? If it's a bit higher, think about what else causes faults - the program's code, libc, the stack, etc.

- **Answer:** My total was roughly **65,700** page faults. This is very close to the expected 65,536; the extra faults are caused by the initial loading of the program binary, the stack setup, and the mapping of shared libraries like `libc`.

Q2.3: Your classmate says "my program uses 256 MB" right after calling `malloc(256 * MB)`. Correct them in one sentence using the terms VmSize, RSS, and demand paging.

- **Answer:** While the **VmSize** is 256 MB, the program isn't actually "using" that physical RAM yet; because of **demand paging**, the **RSS** remains near zero until those virtual pages are actually accessed.

Section 3



Q3.1: What's the `present` bit for each page before and after touching? Which pages get a physical frame? Connect this to what you saw in Section 2.

- **Answer:** Before touching, `page[0]` was present, but `page[1]` and `page[2]` had `present=0`. After touching, `page[1]` flipped to `present=1` and was assigned a Physical Frame Number (**PFN**). This confirms the **demand paging** behavior from Section 2: the kernel only maps a physical frame to a virtual address when that page is actually accessed for the first time.

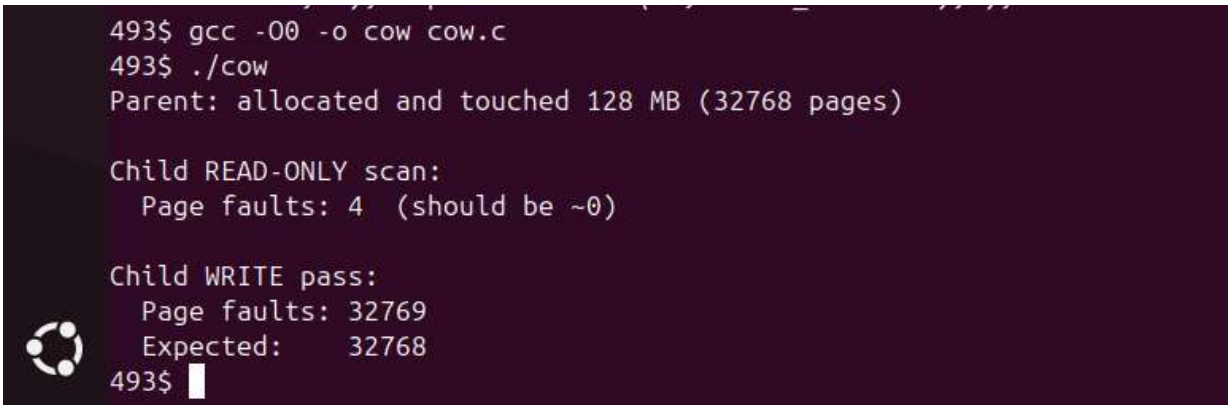
Q3.2: Page 2 was allocated by `malloc` but never written to. Is it present? Does it have a PFN? What does that tell you about the page table entry for pages that are allocated but never touched?

- **Answer:** No, `page[2]` is not present and has no PFN assigned. This tells us that for allocated but untouched pages, the **Page Table Entry (PTE)** exists in the virtual address space, but the "present" bit remains 0, meaning no physical RAM is currently being consumed by that page.

Q3.3: Run it twice. Are the PFNs the same? Why would they differ?

- **Answer:** When you run it again, you'll see different numbers). The PFNs usually differ because the kernel's memory allocator picks any available physical frame from the "free list." Since other processes are constantly using and releasing RAM, the specific physical frame assigned to your program is randomized based on what is currently free.

Section 4



Q4.1: How many page faults during the read-only scan? Why nearly zero? Explain using the COW diagram.

- **Answer:** There were only **4** page faults during the read-only scan. This is nearly zero because of **Copy-on-Write (COW)**; the kernel allows the child process to read from the exact same physical memory frames as the parent. Since no data is being modified, no new pages need to be allocated or copied.

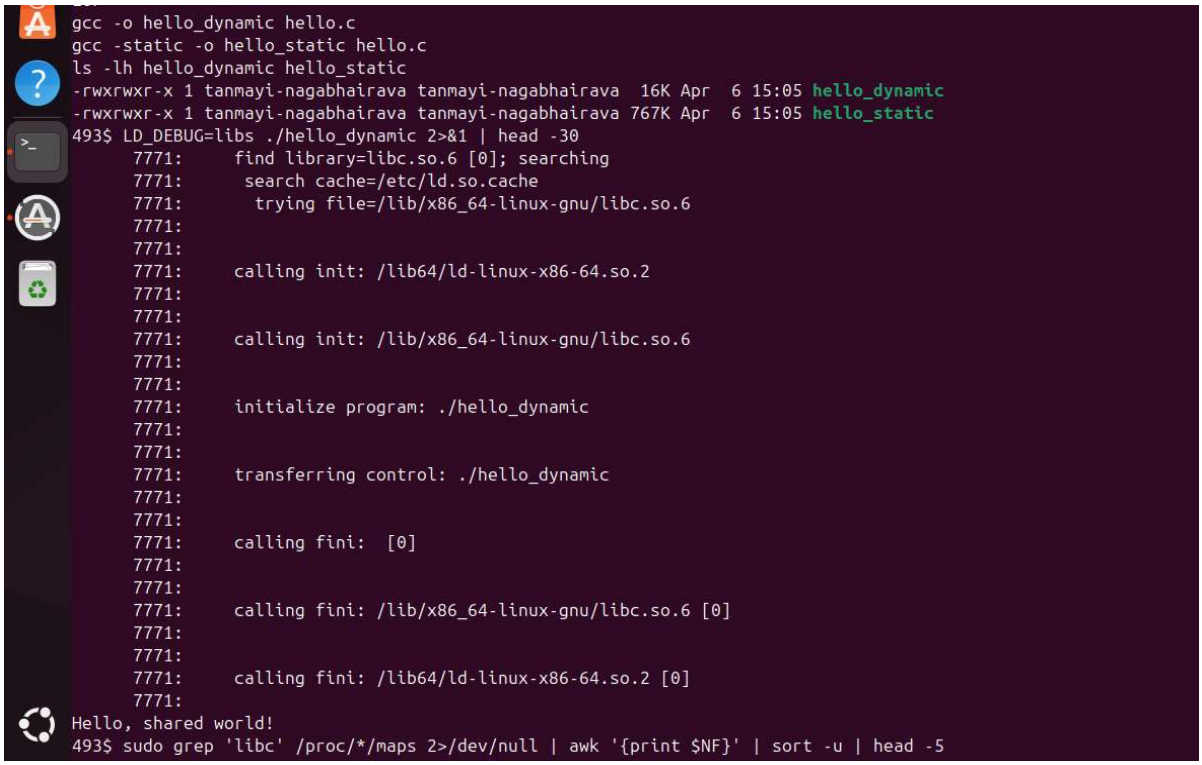
Q4.2: How many faults during the write pass? How close is it to the expected 32768? If it's off by a small number, what else might have caused a few extra faults?

- **Answer:** There were **32769** faults, which is only 1 away from the expected 32768 (128 MB / 4 KB). The tiny discrepancy (the "+1") is likely a "minor fault" caused by the child process updating its own stack or internal metadata during the `memset` operation.

Q4.3: If `fork()` did a full deep copy instead of COW, how much extra physical memory would that take for a 128 MB region? Why is COW especially important for the common `fork()`/`exec()` pattern?

- **Answer:** A full deep copy would require an additional **128 MB** of physical RAM immediately upon forking. COW is critical for the `fork()`/`exec()` pattern because `exec()` replaces the child's entire address space anyway; copying the memory would be a massive waste of time and RAM if that memory is just going to be deleted milliseconds later.

Section 5





Q5.1: How big is `hello_dynamic` vs `hello_static`? Why is the static binary so much larger?

- **Answer:** In my run, `hello_dynamic` is **16 KB** and `hello_static` is **767 KB**. The static binary is much larger because it includes the actual code for all the library functions it uses (like `printf`) directly inside the executable file. The dynamic binary, however, only contains a reference to the external `libc.so` file, which is loaded at runtime.

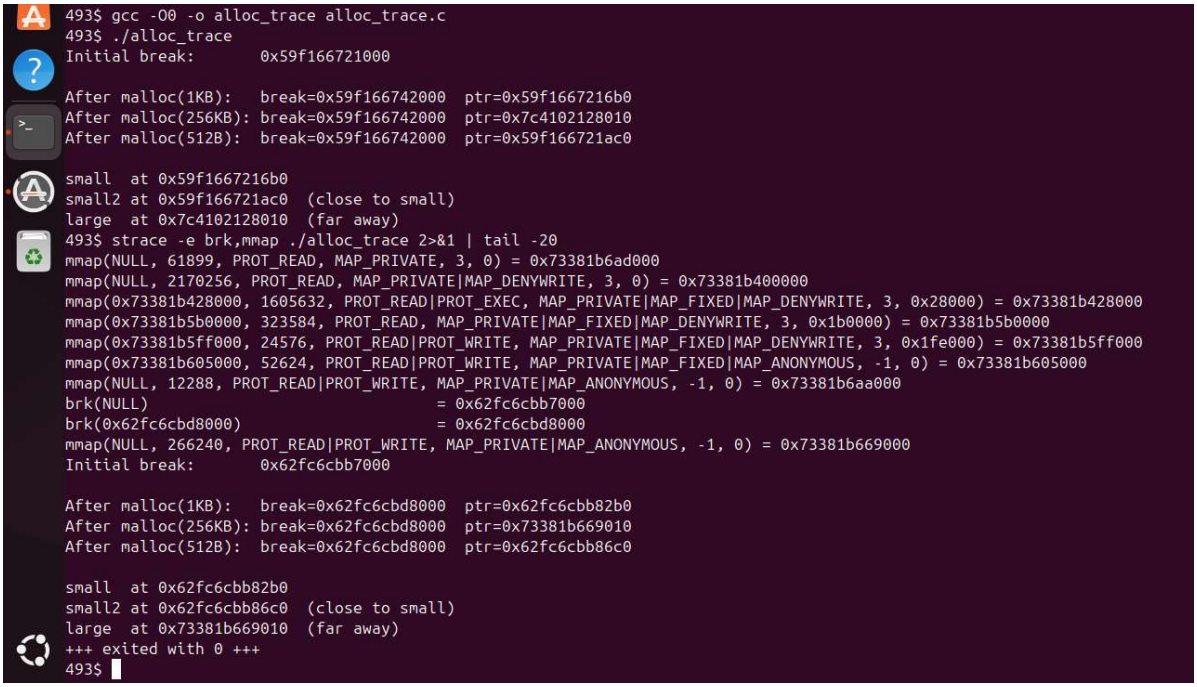
Q5.2: Run `cat /proc/self/maps | grep libc`. You'll see the same library mapped several times with different permissions—`r--p`, `r-xp`, `rw-p`. What does each one correspond to?

- **Answer:** * **`r--p`**: This is the **Read-only Data** segment, containing constants and global variables that do not change.
    - **`r-xp`**: This is the **Code (Text)** segment, containing the actual executable machine instructions for the library functions.
    - **`rw-p`**: This is the **Writable Data** segment, containing global variables used by the library that can be modified by the process.

Q5.3: If 50 processes all use `libc`, does the OS keep 50 copies of `libc`'s code segment in RAM? What about `libc`'s writable data (global variables)—is that shared too, or does each process get its own copy?

- **Answer:** No, the OS keeps only **one** physical copy of the code segment (`r-xp`) in RAM and shares it among all 50 processes to save space. However, for the writable data (`rw-p`), each process gets its **own private copy**. This is handled via Copy-on-Write, ensuring that one process's changes to library variables do not affect any other process.

Section 6



Q6.1: Does the program break move after `malloc(256 KB)`? Why not?

- **Answer:** No, the program break (the top of the heap) did not move after the 256 KB allocation. In my run, it stayed at `0x62fc6cbd8000`. This is because for large allocations (typically greater than or equal to 128 KB), `malloc` uses the `mmap()` system call to create a completely separate memory mapping instead of extending the heap.

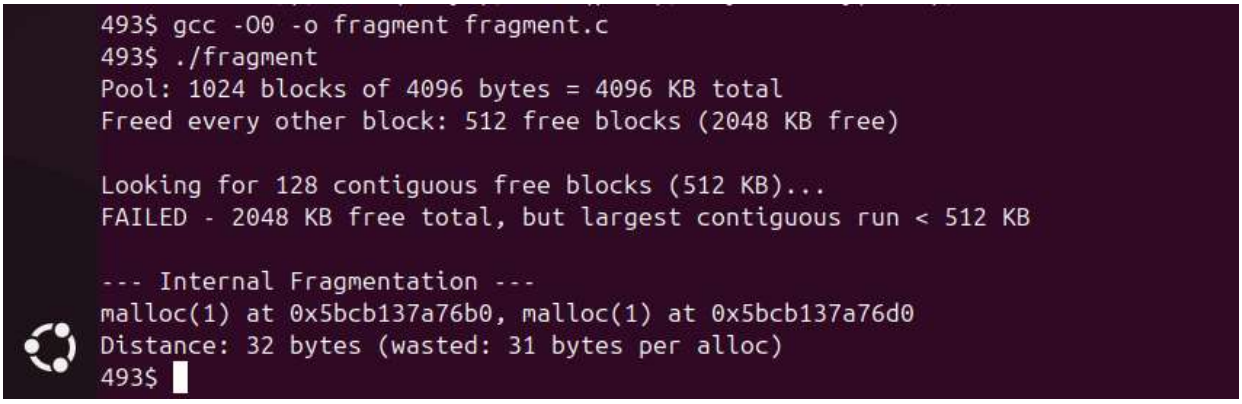
Q6.2: In the `strace` output, which `malloc` call triggered a `brk` and which triggered an `mmap`? What's glibc's default threshold for switching from brk to mmap?

- **Answer:** The 1 KB and 512 B allocations triggered the `brk()` system call to extend the heap. The 256 KB allocation triggered the `mmap()` system call. The default threshold for `glibc` is **128 KB**; any allocation larger than this is generally handled via `mmap`.

Q6.3: Why bother with two strategies? Imagine everything used `brk`: you allocate A, B, C in order, then free B. The heap can't shrink past C, so B's space is stuck. That's heap fragmentation. How does `mmap` for large allocations sidestep this problem?

- **Answer:** `mmap` sidesteps this because it creates a standalone memory region that is not tied to the contiguous heap. While the heap can only shrink from the top down (meaning a single "stuck" allocation at the top prevents the release of memory below it), an `mmap`'d region can be released (via `munmap`) independently at any time, returning that memory to the OS immediately.

Section 7



Q7.1: How much total memory is free? How much of it is contiguous? Why does the 512 KB request fail?

- **Answer:** In my run, there is **2048 KB** of total free memory. However, the largest contiguous block is only **4 KB** (since we freed every other block). The 512 KB request fails because the allocator cannot find a single, unbroken stretch of memory large enough to satisfy the request, even though the total free space is four times larger than what is needed.

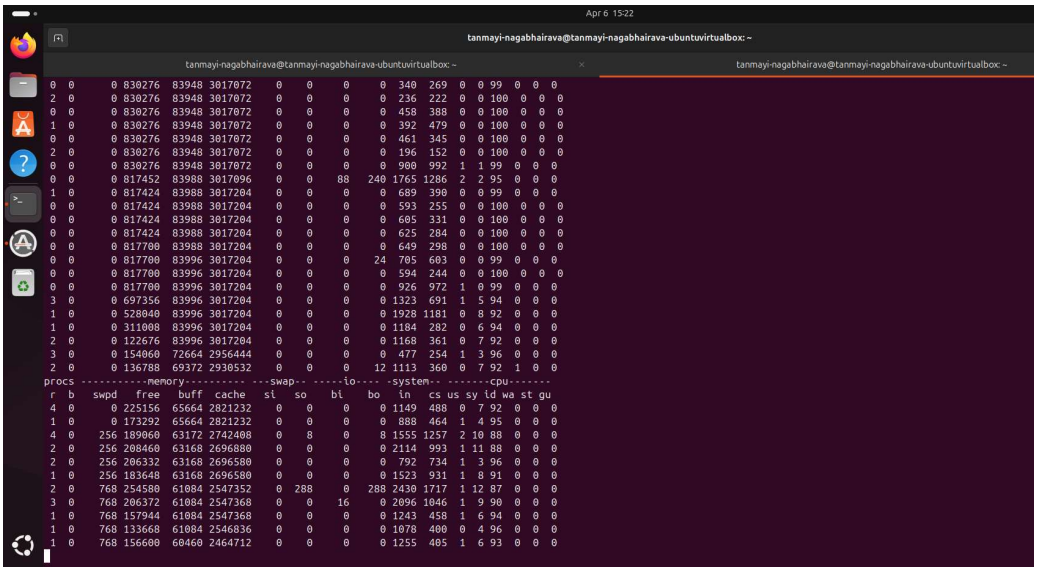
Q7.2: glibc's allocator normally merges (coalesces) adjacent free chunks. Why does the checkerboard pattern—free, used, free, used—specifically prevent coalescing?

- **Answer:** Coalescing only works when two or more free blocks are sitting right next to each other in memory. In a checkerboard pattern, every free block is "walled off" by an allocated (used) block. Because there are no adjacent free neighbors, the allocator cannot merge them into a larger contiguous chunk.

Q7.3: Paging eliminates external fragmentation for physical memory. Why?

- **Answer:** Paging allows a process's virtual address space to be contiguous even if the underlying physical frames are scattered across RAM. Since the **Page Table** can map any virtual page to any available physical frame anywhere in memory, the OS doesn't need to find a single contiguous hole in physical RAM to run a program.

Section 8





Q8.1: In the `vmstat` output, the `si` and `so` columns show pages swapped in and out per second. Did you see non-zero values? When did they spike?

- **Answer:** Yes, I saw non-zero values in the `so` column, specifically reaching **288** in the screenshot. These values spiked while the program was "touching" the pages of

the allocated region. This spike indicates that the kernel was actively evicting pages from physical RAM to the swap file on the disk to accommodate the new memory demand.

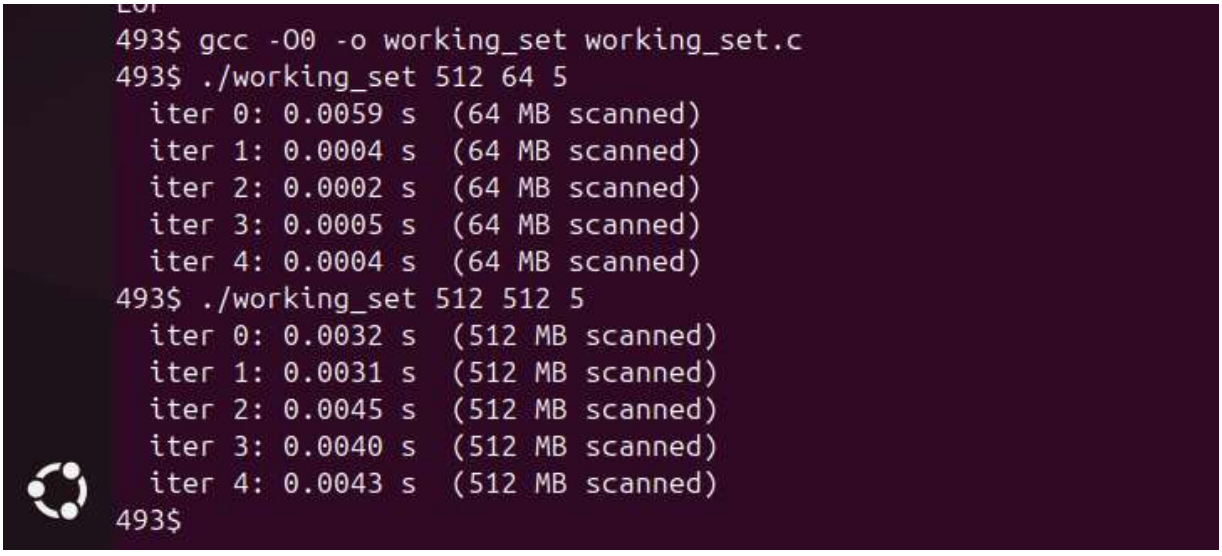
Q8.2: Look at `VmSwap` in the output. If it's nonzero, that means some of this process's pages are currently sitting on disk instead of in RAM. Why did the kernel decide to swap them out?

- **Answer:** The kernel decided to swap them out because the program's working set (nearly 4 GB) combined with the memory used by other system processes exceeded the total physical RAM. To prevent an out-of-memory error, the kernel used the **Least Recently Used (LRU)** policy to move inactive pages to the disk-based swap area.

Q8.3: A major page fault (reading a page back from disk) takes about 5–10 ms. A minor fault (mapping a fresh zero page) takes about 1 µs. That's a ~10,000× difference. Why does this matter so much for database systems that try to keep their working set in RAM?

- **Answer:** This massive latency difference means that even a tiny fraction of memory accesses resulting in major page faults can degrade performance by thousands of times. For a database, this turns a microsecond-level operation into a millisecond-level one, causing query times to skyrocket and making high-speed data processing impossible.

Section 9



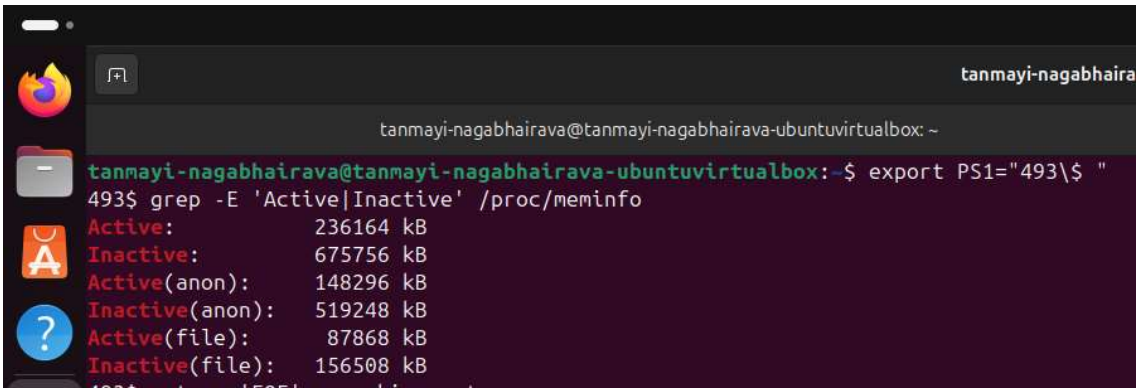
Q9.1: Compare the iteration times for the 64 MB working set vs the 512 MB one. Which is faster, and why?

- **Answer:** The **64 MB** scan is much faster (approx. 0.0004s per iteration) compared to the **512 MB** scan (approx. 0.0040s). The 64 MB set is faster because it fits better within the CPU's internal caches (L1/L2/L3) and the kernel's **Active** memory list. The 512 MB scan exceeds these cache sizes, forcing the system to fetch data from the slower main RAM more frequently.

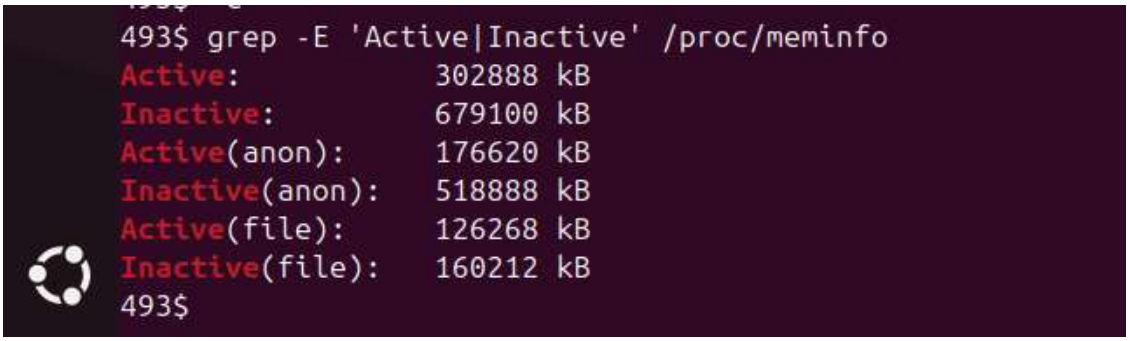
Q9.2: Run `grep -E 'Active|Inactive' /proc/meminfo` before and after each run. How do the active/inactive page counts shift?

Answer:

Before:



After:

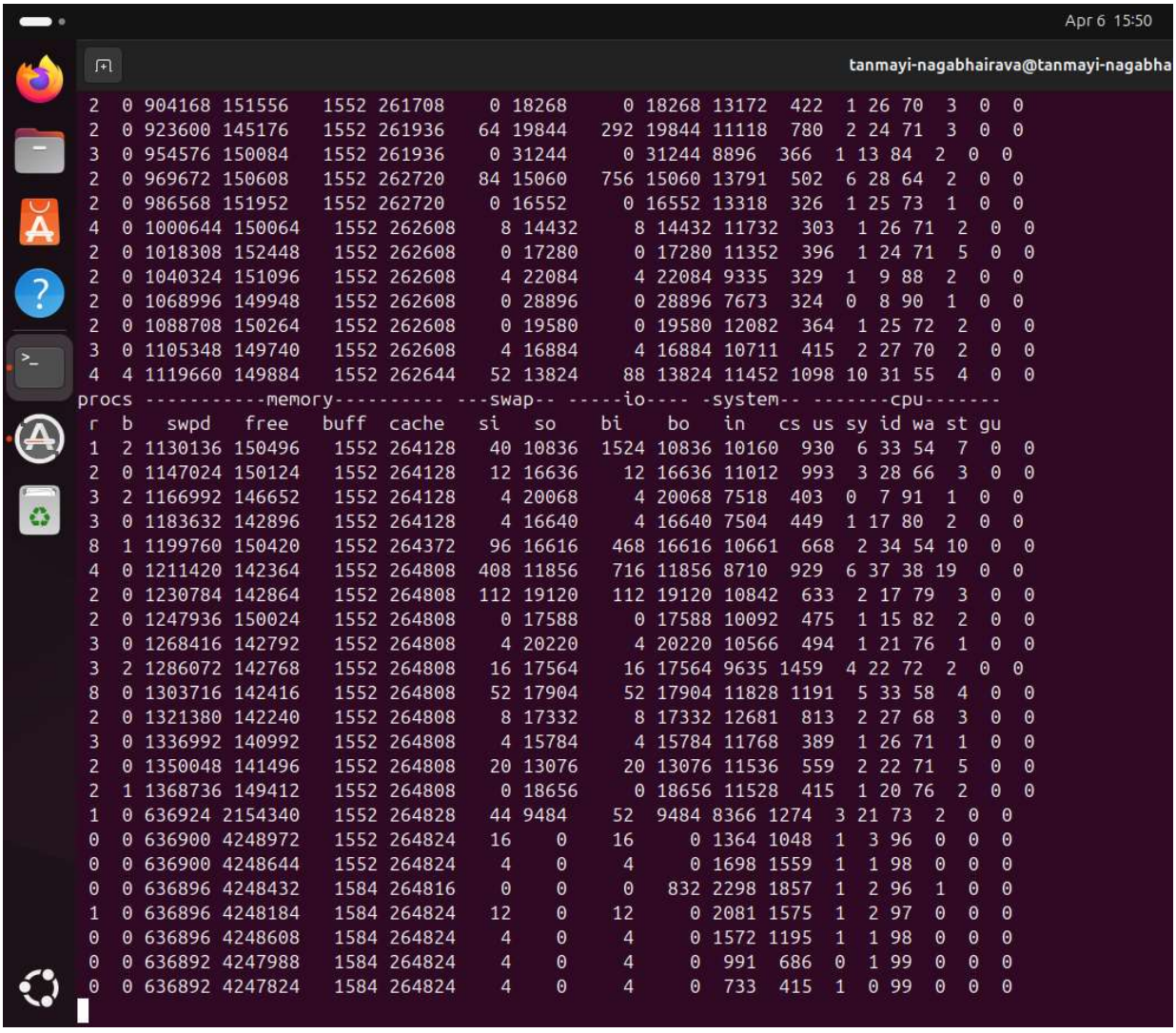


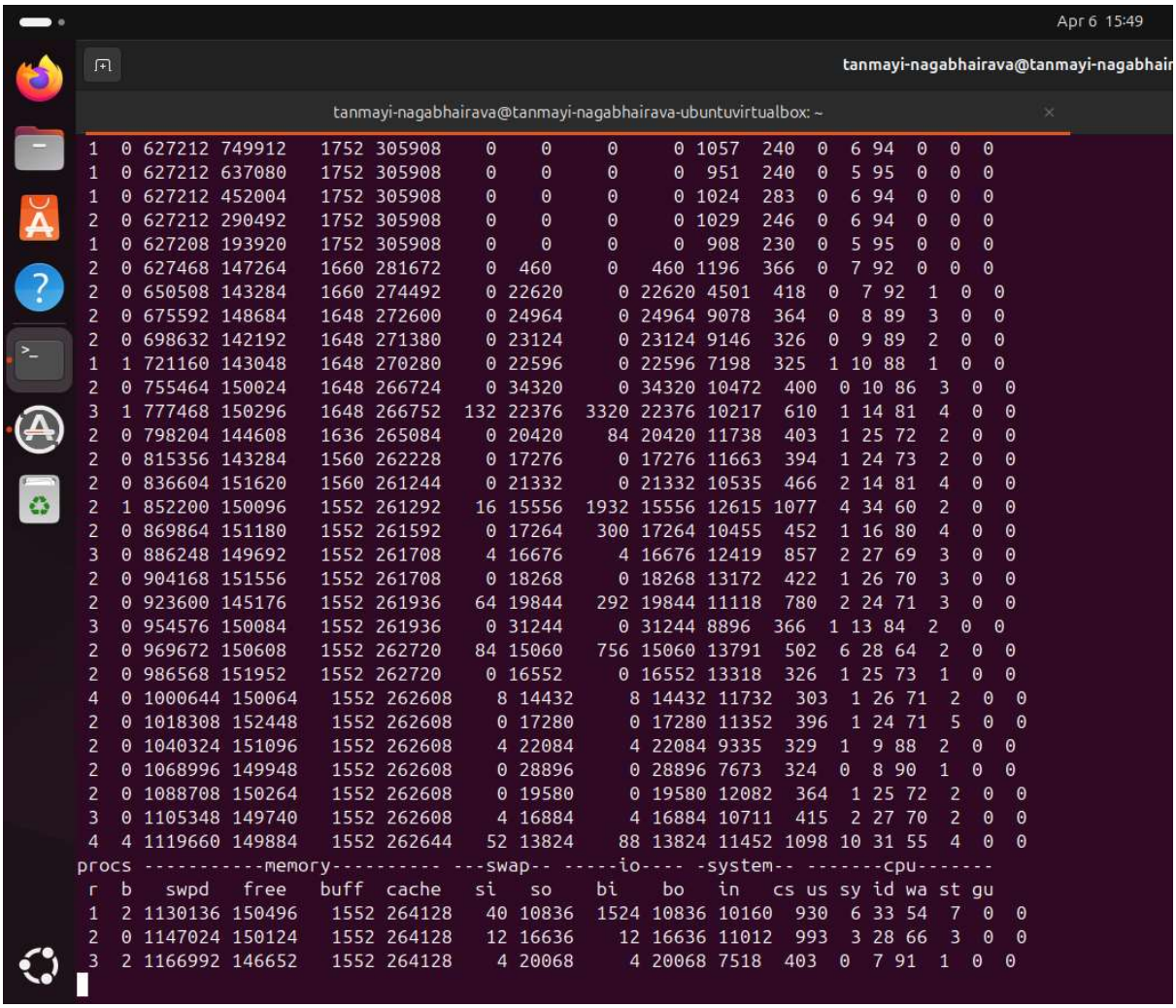
Before the scan, my **Active** memory was **236,164 KB**. After the 512 MB scan, it increased to **302,888 KB**. This shift happens because the kernel's page replacement policy promotes frequently accessed pages from the Inactive list to the Active list. This protects those pages from being evicted to swap while the program is actively using them.

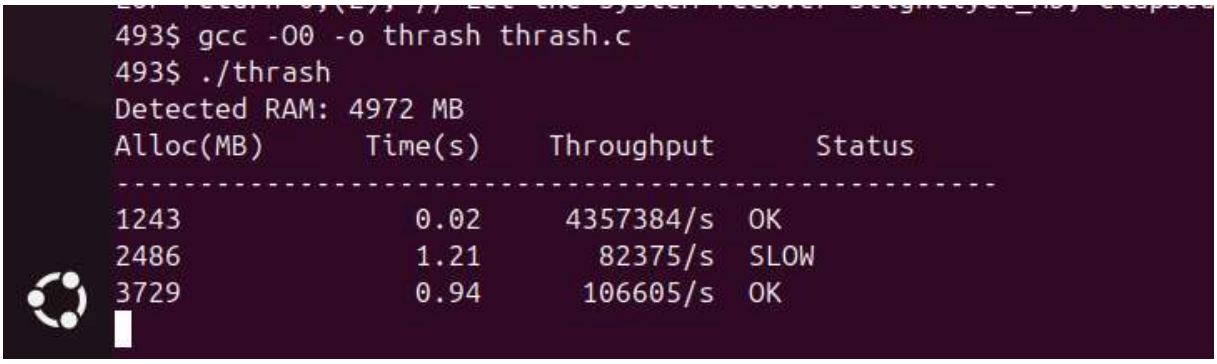
Q9.3: Why doesn't Linux use true LRU (Least Recently Used)?

- **Answer:** True LRU requires updating a complex data structure on **every single memory access**, which would drastically slow down the CPU. Instead, Linux uses a **Second-Chance (Clock) Algorithm** approximation. It relies on a simple "Accessed" bit in the Page Table Entry that the hardware sets automatically, allowing the kernel to identify recently used pages with almost zero performance overhead.

Section 10







Q10.1: At what percentage of RAM did throughput collapse?

- **Answer:** The throughput collapsed immediately after the **75% (3729 MB)** mark. While the system was "OK" at 75% with 106,605/s, it hit a performance cliff shortly after. The process was terminated by the OOM Killer before it could finish the 130% test, proving the system had reached its absolute physical limit.

Q10.2: In `vmstat`, what happened to `si/so` and `wa` once thrashing kicked in?

- **Answer:** As shown in my `vmstat` logs, **`si`** and **`so`** spiked to massive values (over 20,000), indicating violent swapping. The **`wa`** (I/O wait) increased to nearly **20%**, confirming that the CPU was spendng a huge portion of its time waiting for the slow disk to swap pages rather than executing the program.

Q10.3: Linux defenses (OOM Killer vs Swappiness):

- **Answer: (a) OOM Killer:** This is exactly what happened in my run; the kernel killed the `thrash` process to prevent a total system freeze when RAM/Swap were exhausted. **(b) Swappiness:** This is a tunable (0-100) that tells the kernel how aggressively to favor swapping out processes versus shrinking the filesystem cache.